Actividad 2

Tablas ANOVA, pruebas F y t

Autor: Jesús Emiliano Hernández Montalvo

```
# importar las librerías
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np
from scipy import stats

print("Librerías importadas correctamente")
```

```
Librerías importadas correctamente
```

```
# Crear un data frame con los datos a utilizar
datos = pd.DataFrame({
    'Estudiante': range(1, 11),
    'Horas': [1, 2, 3, 4, 5, 6, 7, 8, 9, 10],
    'Nota': [3.2, 4.5, 5.0, 6.8, 7.0, 8.5, 9.0, 9.2, 9.8, 10.0]
})


n = len(datos)                              # Saber la cantidad total de las obse
print(f"Número de observaciones: {n}")

display(datos)
```

```
Número de observaciones: 10
```

	Estudiante	Horas	Nota
0	1	1	3.2
1	2	2	4.5
2	3	3	5.0
3	4	4	6.8
4	5	5	7.0
5	6	6	8.5
6	7	7	9.0
7	8	8	9.2
8	9	9	9.8
9	10	10	10.0

Modelo de Regresión Lineal

Vamos a construir un modelo de regresión lineal para predecir la `Nota` en función de las `Horas` de estudio.

```
#Definir las variables dependiente (y) e independiente (x)
y = datos['Nota']
x = datos['Horas']

# Calcular sus promedios
x_mean = x.mean()
y_mean = y.mean()

# Calcular las covarianzas y varianza de X no normalizadas
Sxy = np.sum((x - x_mean) * (y - x_mean))
Sxx = np.sum((x - x_mean) ** 2)
Syy = np.sum((y - y_mean) ** 2)

# Calcular los betas para el modelo de regresión lineal simple
b1 = Sxy / Sxx
b0 = y_mean - b1 * x_mean

# Valores predichos por el modelo
y_pred = b0 + b1 * x

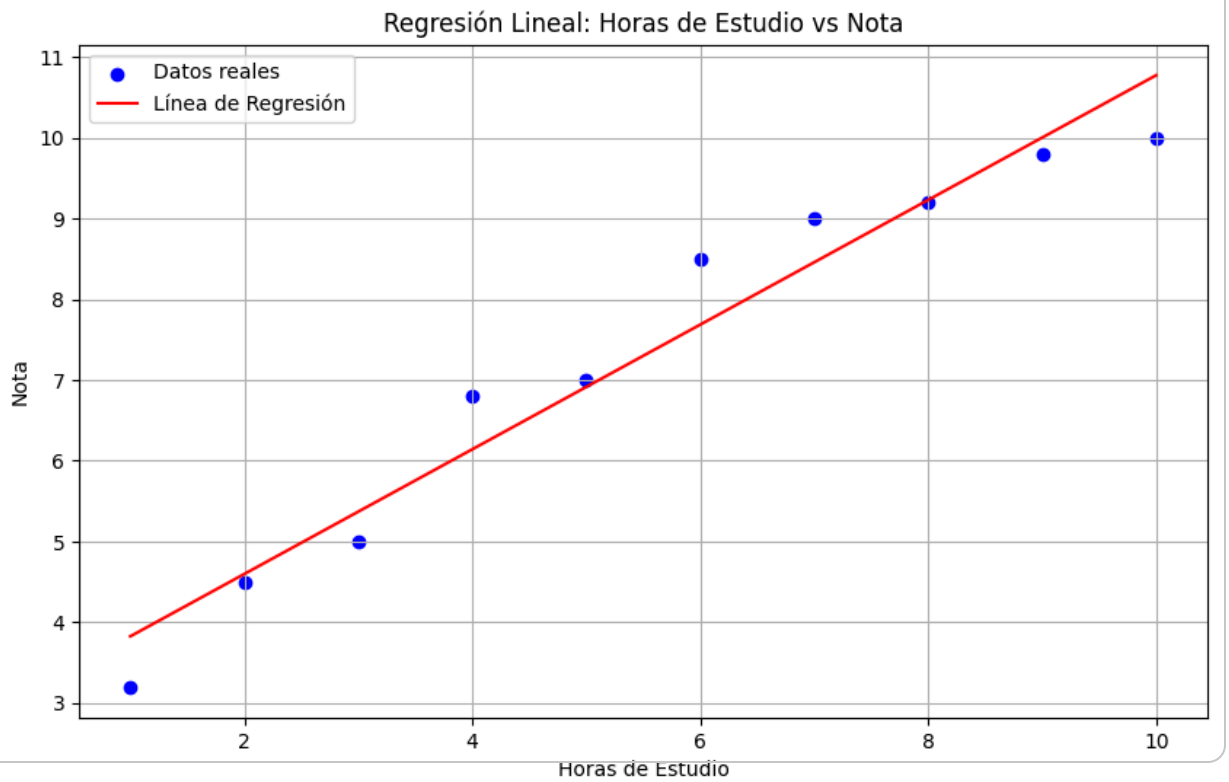
# error de cada observación
residuo = y - y_pred
```

```
# Mostrar los valores
datos['y_pred'] = y_pred
datos['residuo'] = residuo
display(datos)

# Estadísticas descriptivas
print(f"Media de X: {x_mean:.3f}")
print(f"Media de y: {y_mean:.3f}")
print(f"Desviación estándar de X: {np.std(x):.3f}")
print(f"Desviación estándar de y: {np.std(y):.3f}")
print(f"\nb1 (pendiente del modelo): {b1:.3f}")
print(f"b0 (intercepto del modelo): {b0:.3f}")
print(f"\nModelo estimado:  Nota_pred = {b0:.4f} + {b1:.4f} * Horas")
```

Mostrar el resultado oculto

```
plt.figure(figsize=(10, 6))
plt.scatter(x, y, color='blue', label='Datos reales')
plt.plot(x, y_pred, color='red', label='Línea de Regresión')
plt.title('Regresión Lineal: Horas de Estudio vs Nota')
plt.xlabel('Horas de Estudio')
plt.ylabel('Nota')
plt.legend()
plt.grid(True)
plt.show()
```



Gráfica que nos describe el comportamiento de los datos observados y compara con la regresión lineal encontrada de los valores que `Nota` debería de tomar según los coeficientes de β_0 y β_1

```
# Creación de la tabla ANOVA

SST = np.sum((y - y_mean)**2)
SSR = np.sum((y_pred - y_mean)**2)
SSE = np.sum((y - y_pred)**2)


# Calculo de grados de libertad
k = 1     # variable predictora
df_regresion = k
df_error = n - k - 1
df_total = n - 1

# MSR y MSE
MSR = SSR / df_regresion
MSE = SSE / df_error

# Calculo de F
F_statistic = MSR / MSE

# Calculo de p-value
p_value = 1 - stats.f.cdf(F_statistic, df_regresion, df_error)

# Mostrar los resultados de los estadísticos calculados
print(f"Error cuadrático promedio (MSE): {MSE:.6f}")
print(f"SST: {SST:.6f}")
print(f"SSE: {SSE:.6f}")
print(f"SSR: {SSR:.6f}")
print(f"Suma de ambos valores: {SSR + SSE:.6f}")
print(f"F-statistic: {F_statistic:.6f}")
print(f"p-value: {p_value:.6f}")

# Despliegue de la tabla ANOVA
anova = pd.DataFrame({
    'Fuente de variación': ['Regresión', 'Error (Residual)', 'Total'],
    'Suma de cuadrados': [SSR, SSE, SST],
    'Grados de libertad': [df_regresion, df_error, df_total],
    'Cuadrado Medio': [MSR, MSE, ''],
    'F-statistic': [F_statistic, '', ''],
    'p-value': [p_value, '', '']
})

display(anova)
```

Mostrar el resultado oculto

```
# Coeficiente de determinacipon R cuadrado explicada por el modelo
r2 = SSR / SST
print(f"\n Coeficiente de determinación R cuadrada = {r2:.4f}  -> el modelo exp
      f"{r2*100:.2f}% de la variabilidad de la Nota")
```

```
 Coeficiente de determinación R cuadrada = 0.9502  -> el modelo explica el 95.02
```

```
# Definir la zona de rechazo
alpha = 0.05

# Calculo el F crítico y su p_value
F_critical = stats.f.ppf(1 - alpha, df_regresion, df_error)
p_value_F = 1 - stats.f.cdf(F_statistic, df_regresion, df_error)

print("Prueba F (significancia global del modelo)")
print(f"F crítico: {F_critical:.6f}")
print(f"p-value: {p_value_F:.6f}")
print(f"F estadístico: {F_statistic:.6f}")
print("\nNUESTRA HIPÓTESIS")
print(f"H0: b1 = 0   vs   H1: b1 != 0\n")
if F_statistic > F_critical:
    print("F_calculado > F_crítico al tener esta comparación decidimos que se R
    print("Por lo que el modelo es estadísticamente significativo (Horas si pre
else:
    print("F_calculado < F_crítico al tener esta comparación decidimos que no s
```

```
Prueba F (significancia global del modelo)
F crítico: 5.317655
p-value: 0.000002
F estadístico: 152.752906

NUESTRA HIPÓTESIS
H0: b1 = 0   vs   H1: b1 != 0

F_calculado > F_crítico al tener esta comparación decidimos que se RECHAZA H0.
Por lo que el modelo es estadísticamente significativo (Horas si predice Nota).
```

```
# Para cada coeficiente beta se necesita probar los estadísticos y calcular los

SE_b1 = np.sqrt(MSE / Sxx)
SE_b0 = np.sqrt(MSE * (1/n + x_mean**2 / Sxx))

# Estadísticos de prueba
t_b1 = b1 / SE_b1
t_b0 = b0 / SE_b0

# Calculo de p-value de cada beta
p_value_t_b1 = 2 * (1 - stats.t.cdf(abs(t_b1), df_error))
```

```
p_value_t_b0 = 2 * (1 - stats.t.cdf(abs(t_b0), df_error))

# Realizar la prueba de 2 colas
t_critico = stats.t.ppf(1-alpha/2,df_error)

tabla_t = pd.DataFrame({
    'Coeficiente': ['b1(Intercepto)', 'b0(Pendiente)'],
    'Estimado': [b1, b0],
    'Error estándar': [SE_b1, SE_b1],
    't calculado': [t_b1, t_b0],
    'p-value': [p_value_t_b1, p_value_t_b0],
})

print("Significancia individual de los coeficientes por medio de la prueba t")
print(f"t crítico (prueba de dos colas): {t_critico:.6f}")
display(tabla_t)
print("nNuestra hipótesis")
print(f"H0: b1 = 0   vs   H1: b1 != 0\n")
print(f"\nComo p-valor de b1 ({p_value_t_b1:.6e}) < alpha ({alpha}), se rechaza
print(f"Por lo que el modelo es estadísticamente significativo (Horas si predic
```

```
Significancia individual de los coeficientes por medio de la prueba t
t crítico (prueba de dos colas): 2.306004
```

	Coeficiente	Estimado	Error estándar	t calculado	p-value
0	b1(Intercepto)	0.772121	0.062473	12.359325	0.000002
1	b0(Pendiente)	3.053333	0.062473	7.876860	0.000049

```
nNuestra hipótesis
H0: b1 = 0   vs   H1: b1 != 0


Como p-valor de b1 (1.711655e-06) < alpha (0.05), se rechaza H0: b1=0.
Por lo que el modelo es estadísticamente significativo (Horas si predice Nota)
```

```
# Calculo de la relación entre F y t
# Para mostrar que F = t^2
print(f"t_b1^2 = {t_b1**2:.6f}")
print(f"F_calculado = {F_statistic:.6f}")
print(f"¿t_b1^2 == F? -> {np.isclose(t_b1**2, F_statistic)}")
```

```
t_b1^2 = 152.752906
F_calculado = 152.752906
¿t_b1^2 == F? -> True
```

Este resultado confirma la identidad de $F = t^2$ propia de la regresión lineal simple, por lo que se prueba que F global y la pruena t concluyen la misma significancia del modelo.

```
# Calculo de los intervalos de confianza de los parámetros
# Con 95% de confianza

IC_b1 = (b1 - t_critico * SE_b1, b1 + t_critico * SE_b1)
IC_b0 = (b0 - t_critico * SE_b0, b0 + t_critico * SE_b0)

print("Intervalos de confianza de los parámetros")
print(f"IC de b1: {IC_b1[0]:.4f} y {IC_b1[1]:.4f}")
print(f"IC de b0: {IC_b0[0]:.4f} y {IC_b0[1]:.4f}")
```

```
Intervalos de confianza de los parámetros
IC de b1: 0.6281 y 0.9162
IC de b0: 2.1594 y 3.9472
```

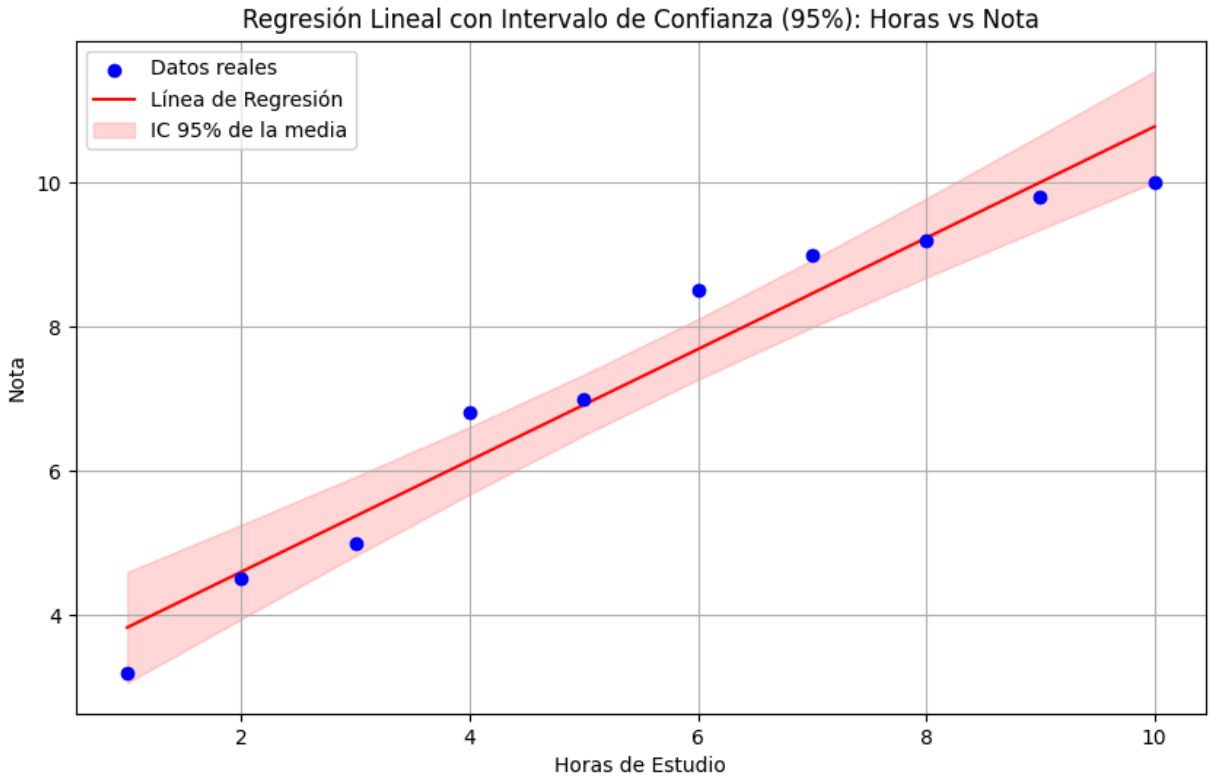
Como se ven estos intervalos de confianza dentro de la gráfica del comportamiento de los datos

```
# Gráfica con Intervalo de Confianza (95%) de la media

# 2) Error estándar de la MEDIA en cada punto de x_linea
#    (misma fórmula que usaste en la tabla de intervalos, pero aplicada a un ra
SE_media = np.sqrt(MSE * (1/n + (x - x_mean)**2 / Sxx))

# 3) Límites del IC 95%
IC_inf = y_pred - t_critico * SE_media
IC_sup = y_pred + t_critico * SE_media

# 4) Graficar: dispersión + línea de regresión + banda sombreada del IC
plt.figure(figsize=(10, 6))
plt.scatter(x, y, color='blue', label='Datos reales',zorder=3)
plt.plot(x, y_pred, color='red', label='Línea de Regresión')
plt.fill_between(x, IC_inf, IC_sup, color='red', alpha=0.15, label='IC 95% de l
plt.title('Regresión Lineal con Intervalo de Confianza (95%): Horas vs Nota')
plt.xlabel('Horas de Estudio')
plt.ylabel('Nota')
plt.legend()
plt.grid(True)
plt.show()
```




```
# Tabla de intervalos de confianza de la Media e intervalos de predicción
# ayuda a calcular el promedio poblacional de Y para un x0
# Al igual que el intervalo de predicción para una nueva observación individual

filas = []
for x0 in x:
    y_hat0 = b0 + b1 * x0
    SE_media = np.sqrt(MSE * (1/n + (x0 - x_mean)**2 / Sxx))
    SE_pred  = np.sqrt(MSE * (1 + 1/n + (x0 - x_mean)**2 / Sxx))

    ic_inf, ic_sup = y_hat0 - t_critico*SE_media, y_hat0 + t_critico*SE_media
    ip_inf, ip_sup = y_hat0 - t_critico*SE_pred,  y_hat0 + t_critico*SE_pred

    filas.append([x0, y_hat0, ic_inf, ic_sup, ip_inf, ip_sup])

tabla_intervalos = pd.DataFrame(
    filas,
    columns=['Horas (x0)', 'Nota_predicha', 'IC_media_inferior', 'IC_media_supe
             'IP_individual_inferior', 'IP_individual_superior']
)
```

```
print("Tabla de Intervalos de Confianza de la Media y Intervalos de Predicción"
print(tabla_intervalos.to_string(index=False))
print("\nNota: el IP siempre es más ancho que el IC de la media, porque además
print("la incertidumbre sobre dónde está la recta verdadera, incorpora la")
print("variabilidad individual de una observación futura.")
```

```
Tabla de Intervalos de Confianza de la Media y Intervalos de Predicción
 Horas (x0)  Nota_predicha  IC_media_inf  IC_media_sup  IP_individual_inf  IP_in
          1       3.825455      3.056371      4.594538           2.307662
          2       4.597576      3.945305      5.249846           3.135501
          3       5.369697      4.821123      5.918270           3.950846
          4       6.141818      5.675002      6.608634           4.752530
          5       6.913939      6.493929      7.333950           5.539671
          6       7.686061      7.266050      8.106071           6.311792
          7       8.458182      7.991366      8.924998           7.068894
          8       9.230303      8.681730      9.778877           7.811452
          9      10.002424      9.350154     10.654695           8.540350
         10      10.774545     10.005462     11.543629           9.256753

Nota: el IP siempre es más ancho que el IC de la media, porque además de
la incertidumbre sobre dónde está la recta verdadera, incorpora la
variabilidad individual de una observación futura.
```

Se puede observar que el intervalo de predicción es más ancho que el intervalo de confianza ya que la incertidumbre incorpora la variabilidad de la observación futura.

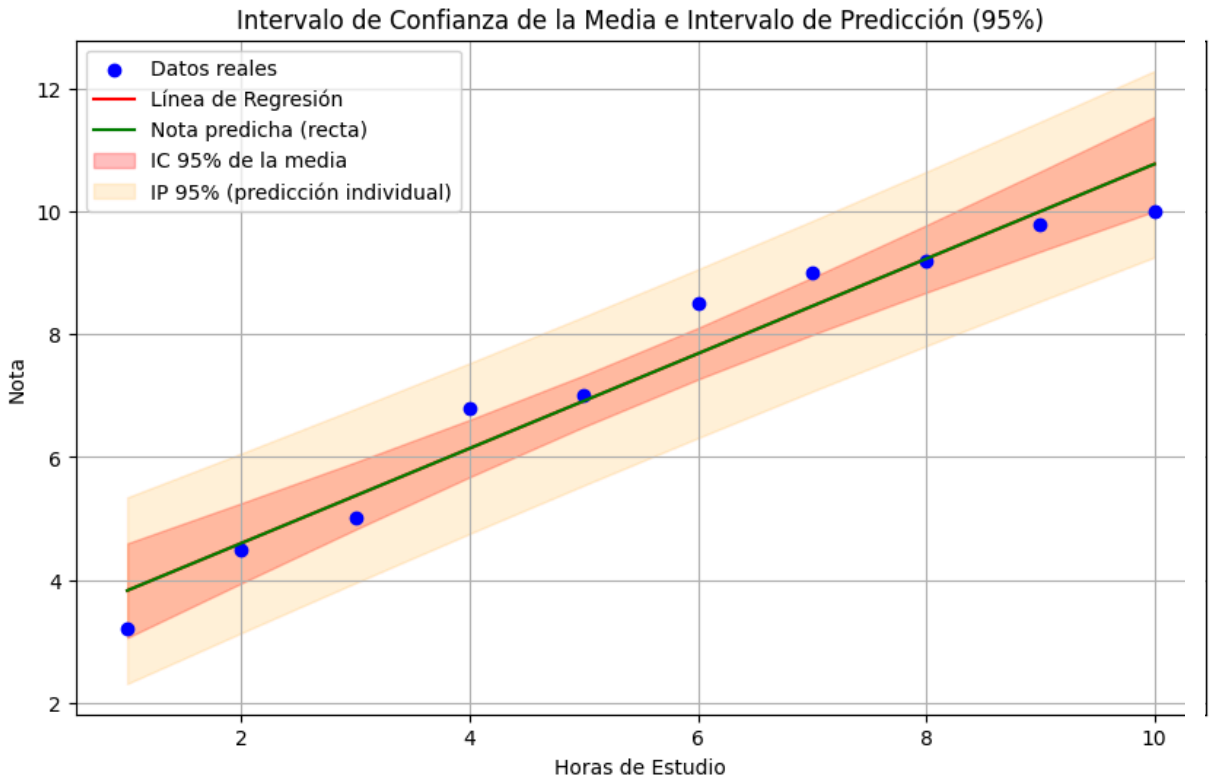
```
# Graficar los IC de la media individual e IP individual utilizando la tabla a

fig, ax = plt.subplots(figsize=(10, 6))

# Datos reales
plt.scatter(x, y, color='blue', label='Datos reales',zorder=3)
plt.plot(x, y_pred, color='red', label='Línea de Regresión')
ax.plot(tabla_intervalos['Horas (x0)'], tabla_intervalos['Nota_predicha'],
        color='green', label='Nota predicha (recta)', zorder=3)
ax.fill_between(tabla_intervalos['Horas (x0)'],
                tabla_intervalos['IC_media_inf'],
                tabla_intervalos['IC_media_sup'],
                color='red', alpha=0.25, label='IC 95% de la media')
ax.fill_between(tabla_intervalos['Horas (x0)'],
                tabla_intervalos['IP_individual_inf'],
                tabla_intervalos['IP_individual_sup'],
                color='orange', alpha=0.15, label='IP 95% (predicción indivi

ax.set_title('Intervalo de Confianza de la Media e Intervalo de Predicción (9
ax.set_xlabel('Horas de Estudio')
ax.set_ylabel('Nota')
ax.legend()
```

```
ax.grid(True)
```


Intervalo de Confianza de la Media e Intervalo de Predicción (95%)
Datos reales
Línea de Regresión
Nota predicha (recta)
IC 95% de la media
IP 95% (predicción individual)
Nota
Horas de Estudio
2
4
6
8
10
12